

Population Effect Estimation in Bayesian Hierarchical Models Using Sum-to-Zero Constraints

StanCon 2026

Sean Pinkney

sean.pinkney@gmail.com

SVP Measurement Analytics at
Publicis Groupe

Stan Developer

Nikolai Vetr

nikgvetr@stanford.edu

Department of Pathology,
Stanford University



Slides and code at



github.com/spinkney/open-talks

Agenda

- What is a sum-to-zero constraint?
- Hierarchical models between frequentist and bayesian approaches
- How is it possible to translate between them?
- Implications

What is a sum-to-zero constraint?

It's simply

$$\sum_j^n a_j = 0$$

To construct one might randomly draw i.i.d. $n - 1$ values y and

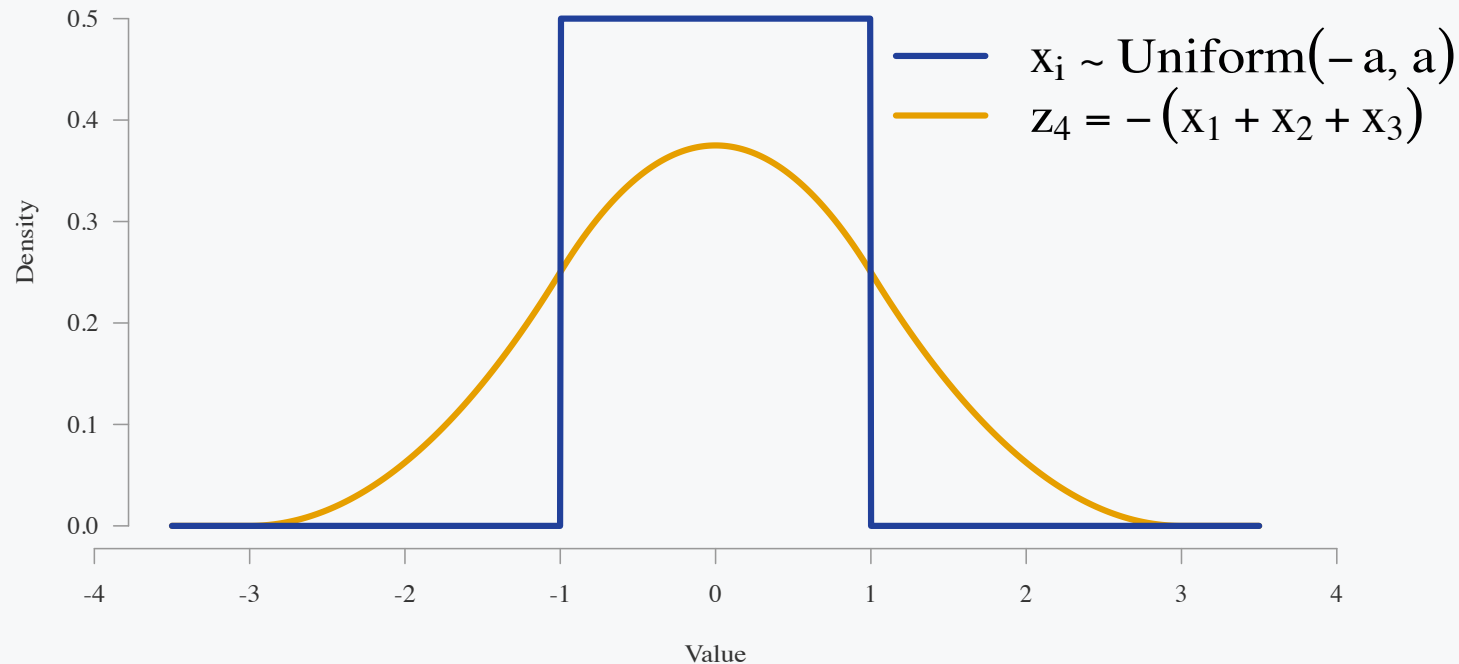
$$y_n \stackrel{\text{set}}{=} - \sum_i^{n-1} y_i$$

$$\mathbf{z} = [y_1 \ y_2 \ \dots \ y_{n-1} \ y_n]$$

$$\sum_i^n z_i = 0$$

What is a sum-to-zero constraint?

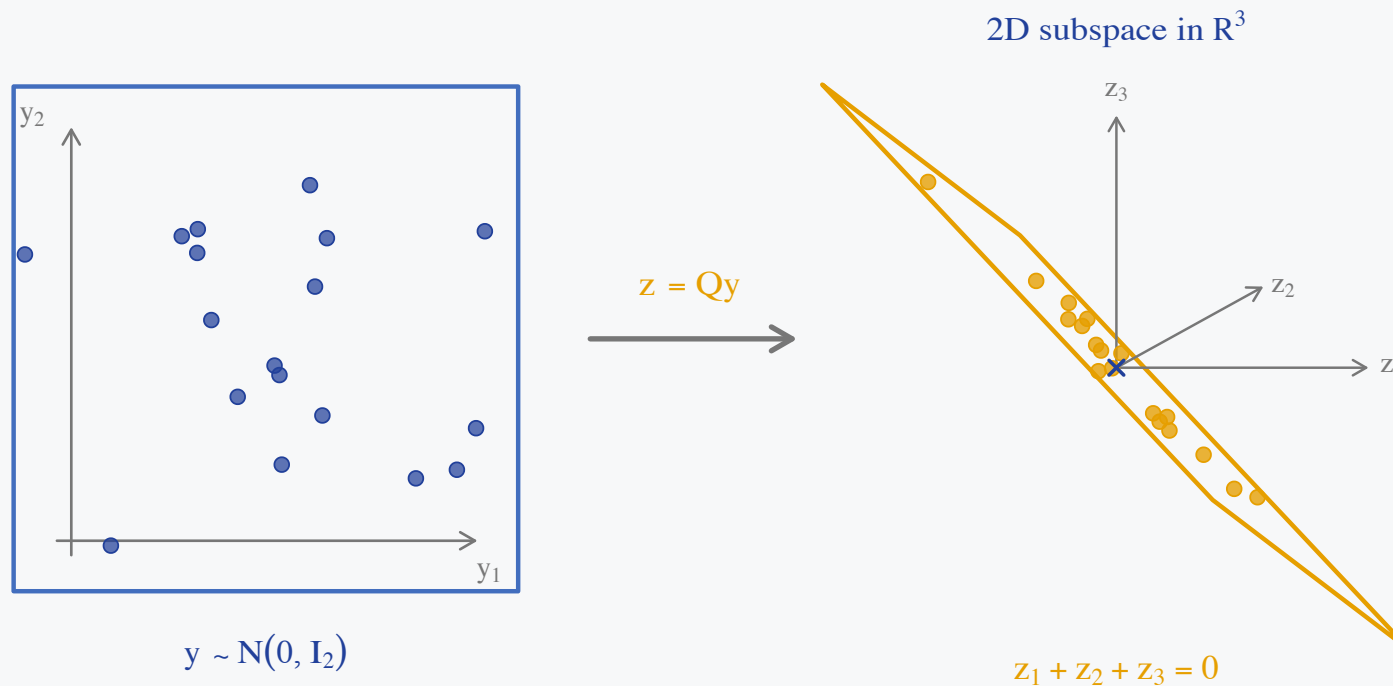
Since only z_n is a function of y the marginal density will be the density of the sum of $z_{i:n-1}$. Most modelers would not want this!



The implicit prior on the last element is clearly different. Furthermore, the covariance of the final element is coupled strongly with the other elements.

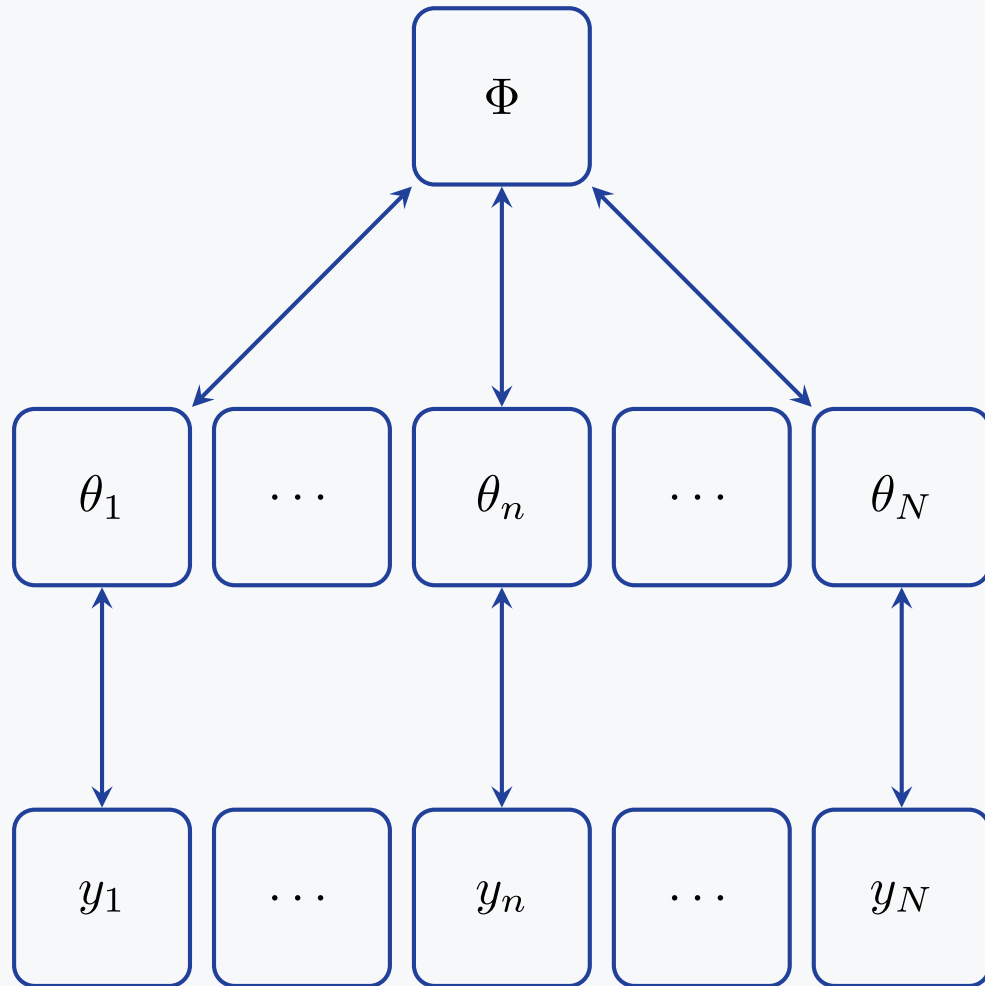
What is a sum-to-zero constraint?

We can balance the density across the values by using an orthonormal basis for the zero-sum subspace, $QQ^\top = P_0 = I_n - \frac{1}{n}\mathbf{1}_n\mathbf{1}_n^\top$.



All the values are now isotropic and exchangeable.

Bayesian hierarchical models



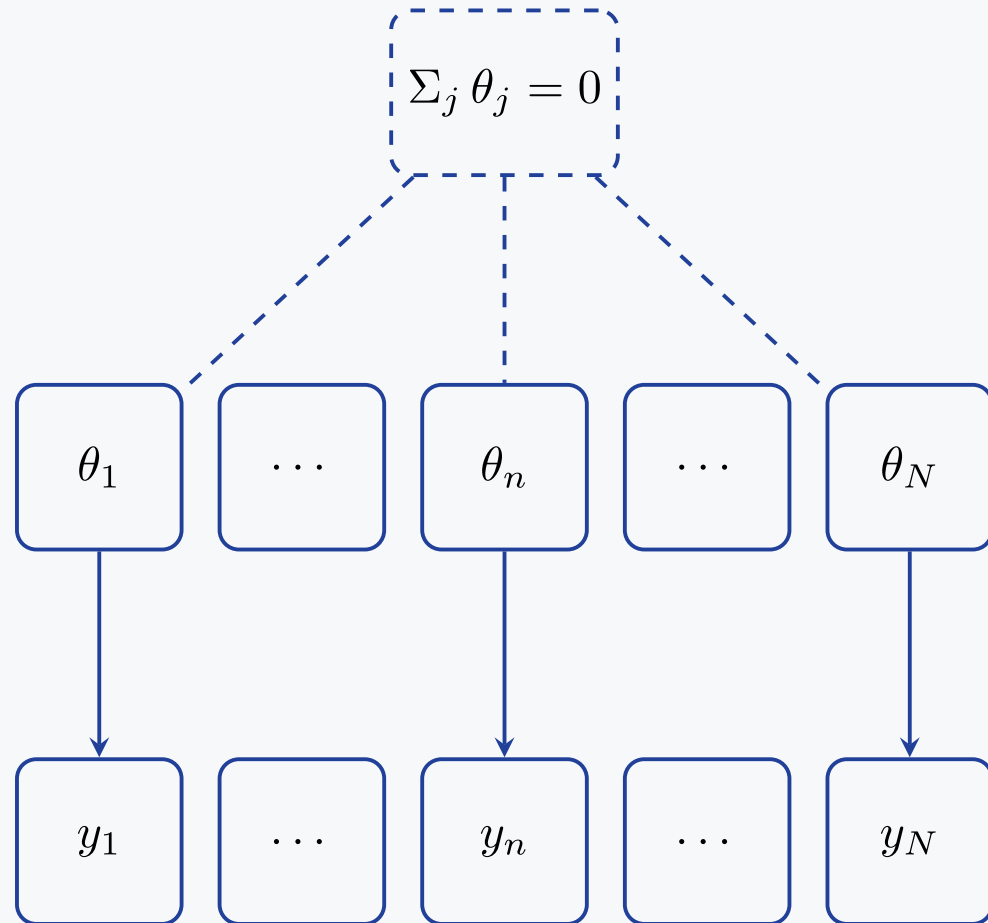
Sharing of information happens

- Φ population distribution
- Globally
- Bi-directionally
- Partial pooling

When the evidence or data for a parameter are

- low
 - estimate is closer to prior
- large
 - data swamps prior
 - prior pull is weak

Frequentist hierarchical models



No population distribution Φ

- the θ_j are **fixed unknowns**, not draws
- no pooling: each θ_j is estimated from group j data alone
- information flows one way, $\theta_j \rightarrow y_j$

Groups are tied together only by a *hard* constraint

- e.g. effects coding $\sum_j \theta_j = 0$
- identifies the grand mean instead of a population parameter

What is the population effect?

Let's take a 2-level, varying slopes, varying intercept model with i units and j groups

$$y_{ij} = \underbrace{\alpha + a_j}_{\text{varying intercept}} + X \underbrace{(\beta + b_j)}_{\text{varying slope}} + \epsilon_{ij}$$

population parameters

Group-level (conditional): $E(y_{ij} \mid X, j) = \alpha + a_j + X(\beta + b_j)$

Population-level (marginal): $E(y_{ij} \mid X) = \alpha + X\beta$

when $E(a_j) = E(b_j) = 0$

Different populations

The data inform the group coefficients α_j and β_j whereas the population α and β sit one level up.

Bayesian hierarchical model

α and β are *population* parameters and the observed groups are **draws from** that population

$$a_j \sim \mathcal{N}(\alpha, \sigma_a), \quad b_j \sim \mathcal{N}(\beta, \sigma_b)$$

A benefit is that new groups may still be inferred from the population distribution as any finite draw of J groups has

$$\bar{a} = \frac{1}{J} \sum_j a_j \neq \alpha$$

Frequentist effects coding

No population to draw from — the J observed groups **are** the population

The “population” parameter is the finite-population grand mean

$$\alpha = \frac{1}{J} \sum_j (\alpha + a_j) \iff \sum_j a_j = 0$$

Is there an equivalence?

Suppose we restrict to varying intercepts, $x_i \sim \mathcal{N}(\alpha + a_j, \sigma_x)$:

Bayesian hierarchical model

$$a_j^{\text{bayes}} \sim \mathcal{N}(0, \sigma_a)$$

$$\bar{a} = \frac{1}{J} \sum_j a_j^{\text{bayes}} \sim \mathcal{N}\left(0, \frac{\sigma_a^2}{J}\right)$$

Frequentist effects coding

$$\alpha = \frac{1}{J} \sum_j (\alpha + a_j^{\text{s2z}}) \iff \sum_j a_j^{\text{s2z}} = 0$$

Take

$$\alpha_{\text{s2z}} \stackrel{\text{set}}{=} \alpha_{\text{bayes}} + \bar{a}$$

$$a_j^{\text{s2z}} \stackrel{\text{set}}{=} a_j^{\text{bayes}} - \bar{a}$$

$$\sum a_j^{\text{s2z}} = 0$$

Does it work?

Let's simulate six deliberately unbalanced groups from

$$x_i \sim \mathcal{N}(\alpha + a_{j[i]}, \sigma_x), \quad a_j \sim \mathcal{N}(0, \tau^2),$$

with

```
alpha = 5  
tau = 2.5  
sigma_x = 1  
n_j = (8, 13, 21, 34, 55, 89)
```

For this finite draw,

$$\frac{1}{J} \sum_j a_j^{\text{bayes}} = \bar{a} = 0.85, \quad \alpha_{s2z} = 5.85.$$

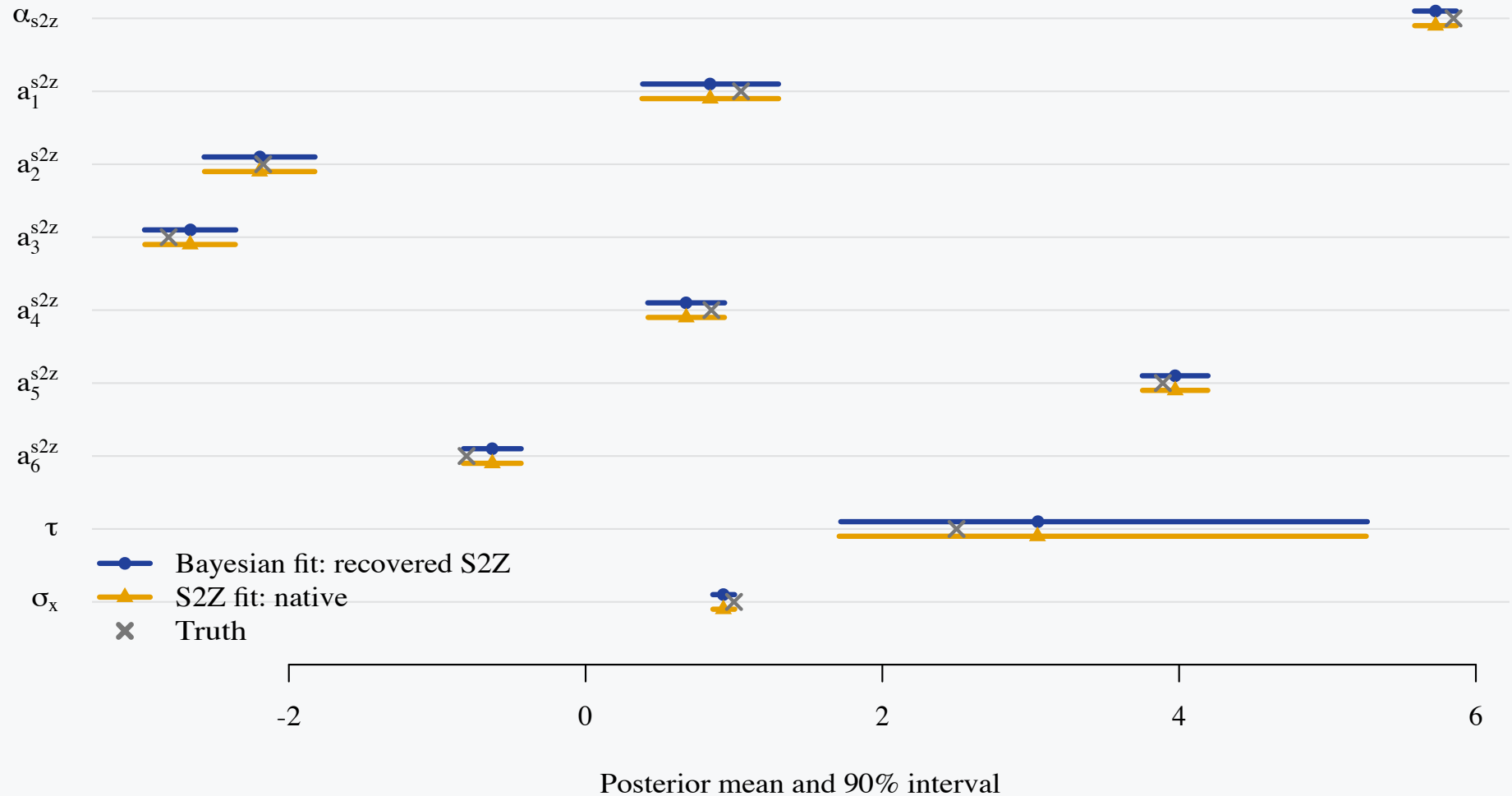
Sampling 10k iterations over 4 chains. The first Stan program samples unconstrained Bayesian varying effects; the second samples a `sum_to_zero_vector` directly.

Conversion

Bayesian specification and conversion to sum-to-zero equivalent

```
1 data {
2   int<lower=1> N;
3   int<lower=2> J;
4   array[N] int<lower=1, upper=J> group;
5   vector[N] x;
6   real<lower=0> alpha_prior_sd;
7 }
8 parameters {
9   real alpha_bayes;
10  vector[J] a_bayes;
11  real<lower=0> tau;
12  real<lower=0> sigma_x;
13 }
14 model {
15   alpha_bayes ~ normal(0, alpha_prior_sd);
16   a_bayes ~ normal(0, tau);
17   tau ~ normal(0, 5);
18   sigma_x ~ normal(0, 2);
19   x ~ normal(alpha_bayes + a_bayes[group], sigma_x);
20 }
21 generated quantities {
22   real mean_a = mean(a_bayes);
23   real alpha_s2z_recovered = alpha_bayes + mean_a;
24   vector[J] a_s2z_recovered = a_bayes - mean_a;
25 }
```

The two fits agree



Across α_{s2z} and the six a_j^{s2z} , the largest posterior-mean difference is 0.002. Across every plotted parameter, the largest difference is 0.003: Monte Carlo error at the scale of the plot.

What about going from sum-to-zero to population models?

Given τ , let $v_m = \tau^2/J$, with $\alpha_{\text{bayes}} \perp \bar{a} \mid \tau$. Then

$$\begin{aligned} \alpha_{\text{bayes}} &\sim \mathcal{N}(0, s_\alpha^2), & \bar{a} \mid \tau &\sim \mathcal{N}(0, v_m), \\ \alpha_{s2z} = \alpha_{\text{bayes}} + \bar{a} &\implies \begin{cases} \text{Var}(\alpha_{s2z} \mid \tau) = s_\alpha^2 + v_m, \\ \text{Cov}(\bar{a}, \alpha_{s2z} \mid \tau) = v_m, \end{cases} \\ \begin{pmatrix} \bar{a} \\ \alpha_{s2z} \end{pmatrix} \mid \tau &\sim \mathcal{N} \left[\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} v_m & v_m \\ v_m & s_\alpha^2 + v_m \end{pmatrix} \right]. \end{aligned}$$

The conditional-normal formula therefore gives

$$\begin{aligned} \bar{a} \mid \alpha_{s2z}, \tau &\sim \mathcal{N}(w\alpha_{s2z}, v_c), \\ \underbrace{\frac{v_m}{s_\alpha^2 + v_m}}_w, & \quad \underbrace{v_m - \frac{v_m^2}{s_\alpha^2 + v_m}}_{v_c}. \end{aligned}$$

What about going from sum-to-zero to population models?

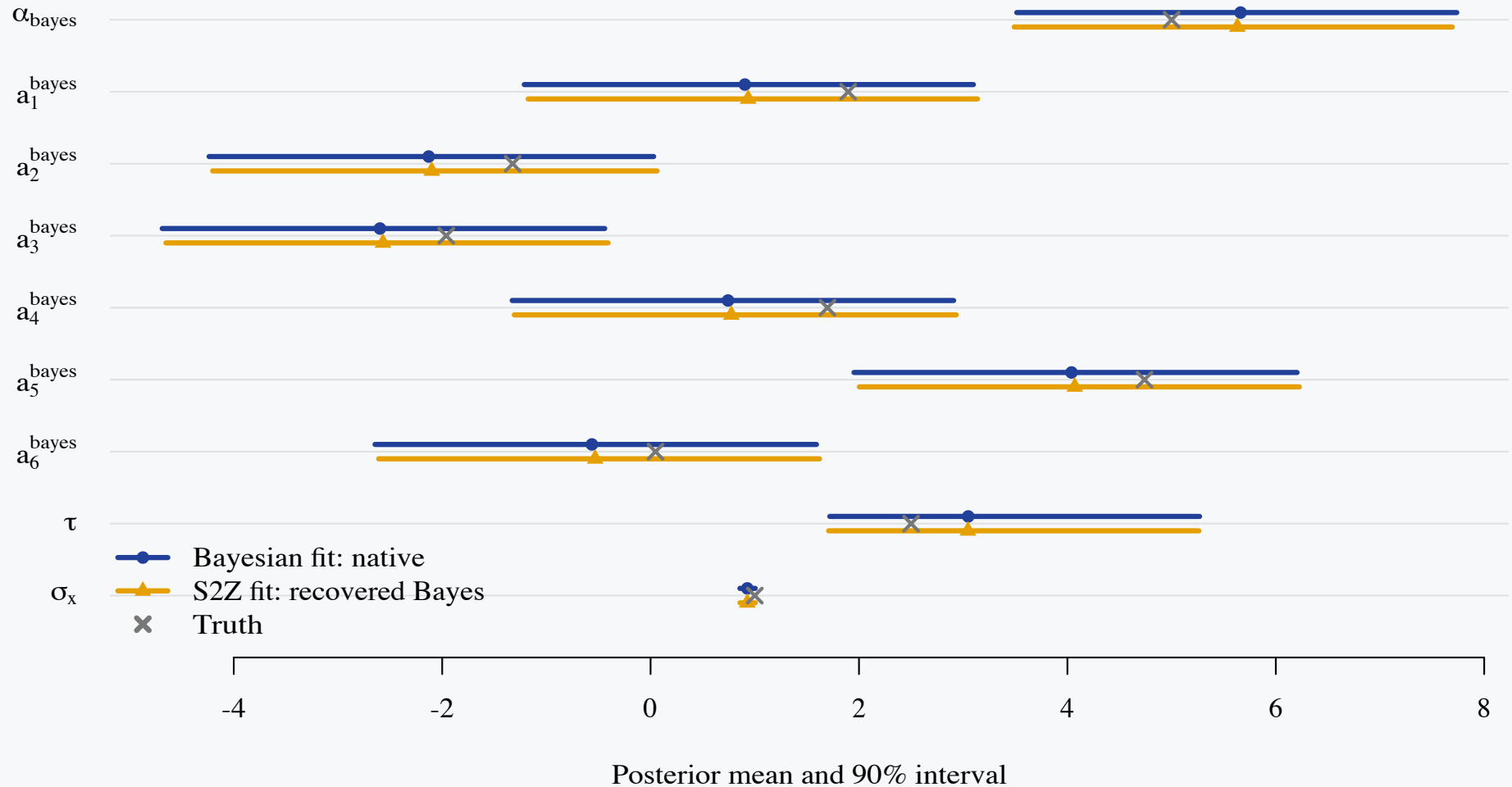
For each sum-to-zero posterior draw, sample the missing realized mean and shift the coordinates:

$$m \mid \alpha_{s2z}, \tau \sim \mathcal{N}(w\alpha_{s2z}, v_c), \quad \alpha_{\text{bayes}} = \alpha_{s2z} - m, \quad a_j^{\text{bayes}} = a_j^{s2z} + m,$$

where $\mathcal{N}(\text{mean}, \text{variance})$ is used.

```
1 parameters {
2   real alpha_s2z; real<lower=0> tau, sigma_x;
3   sum_to_zero_vector[J] a_s2z;
4 }
5 model {
6   tau ~ normal(0, 5); sigma_x ~ normal(0, 2);
7   alpha_s2z ~ normal(0, hypot(alpha_prior_sd, tau / sqrt(J)));
8   a_s2z ~ normal(0, tau);
9   target += log(tau);
10  x ~ normal(alpha_s2z + a_s2z[group], sigma_x);
11 }
12 generated quantities {
13   real mean_a_bayes, alpha_bayes_recovered;
14   vector[J] a_bayes_recovered;
15   {
16     real var_mean_a = square(tau) / J;
17     real var_alpha = square(alpha_prior_sd);
18     real weight = var_mean_a / (var_alpha + var_mean_a);
19     real conditional_sd = sqrt(var_alpha * var_mean_a / (var_alpha + var_mean_a));
20     mean_a_bayes = normal_rng(weight * alpha_s2z, conditional_sd);
21   }
22   alpha_bayes_recovered = alpha_s2z - mean_a_bayes;
23   a_bayes_recovered = a_s2z + mean_a_bayes;
24 }
```

Back to Bayesian: the two fits agree



Across α_{bayes} and the six a_j^{bayes} , the largest posterior-mean difference is 0.032. Across every plotted parameter, the largest difference is 0.032. The largest standardized gap is 1.6 combined Monte Carlo SEs.

ESS and ESS/grad on the two parameterizations

Parameter	Bayesian		Sum-to-zero	
	ESS	ESS/grad	ESS	ESS/grad
α	5,557	0.002	33,461	0.121
a_1	5,719	0.002	39,725	0.144
a_2	5,683	0.002	64,383	0.234
a_3	5,650	0.002	71,686	0.260
a_4	5,588	0.002	56,372	0.204
a_5	5,581	0.002	46,215	0.168
a_6	5,572	0.002	39,120	0.142
τ	11,574	0.004	57,662	0.209
σ_x	16,549	0.006	61,344	0.223

4 chains $\times$ 10,000 sampling iterations = 40,000 post-warmup draws, plus 4,000 warmup iterations per chain. Bulk ESS is pooled across chains. ESS/grad divides by the total post-warmup `n_leapfrog__`: Bayesian = 2,796,128; sum-to-zero = 275,672.

Idea on one slide

Super-population decomposition

For every draw from the induced-prior sum-to-zero fit, draw the missing common shift from the prior-implied conditional distribution. This restores the Bayesian super-population draw.

The native Bayesian and sum-to-zero estimates are different. Equivalence is distributional and holds only after randomly augmenting the sum-to-zero draws.

Extends to varying prior standard deviations on the varying effects. Common in Bayesian models and we can derive similar logic.

$$\underbrace{\alpha_{s2z}}_{\text{mean of these } J \text{ groups}} = \underbrace{\alpha_{\text{bayes}}}_{\text{super-population mean}} + \underbrace{\frac{1}{J} \sum a_j^{\text{bayes}}}_{\bar{a}},$$

$$\bar{a} \mid \tau \sim \mathcal{N}(0, \tau^2 / J)$$

$$m \mid \alpha_{s2z}, \tau \sim \mathcal{N}(w\alpha_{s2z}, v_c),$$

$$\alpha_{\text{Bayes}}^{\text{rec}} = \alpha_{s2z} - m,$$

$$\mathbf{a}_{\text{Bayes}}^{\text{rec}} = \mathbf{a}_{s2z} + m\mathbf{1}.$$

$$(\alpha_{\text{Bayes}}^{\text{rec}}, \mathbf{a}_{\text{Bayes}}^{\text{rec}}) \mid \mathbf{y} \stackrel{d}{=} (\alpha_{\text{Bayes}}, \mathbf{a}_{\text{Bayes}}) \mid \mathbf{y},$$

- Repeat the decomposition on the log-SD scale:
 $\log s_j = \gamma_{s2z} + b_j^{s2z}, \quad \mathbf{1}^\top \mathbf{b}_{s2z} = 0,$
 then restore $\bar{b}$.

From one common SD to group-specific SDs

The original model used one common prior SD τ for every varying intercept. We now learn a separate prior SD s_j for each group:

$$a_j \mid \tau \sim \mathcal{N}(0, \tau^2) \quad \implies \quad a_j \mid s_j \sim \mathcal{N}(0, s_j^2), \quad j = 1, \dots, J.$$

The J positive scales are partially pooled through a log-scale hierarchy:

$$\gamma \sim \mathcal{N}(\mu_\gamma, s_\gamma^2), \quad b_j \mid \omega \sim \mathcal{N}(0, \omega^2), \quad s_j = \exp(\gamma + b_j).$$

Here $\exp(\gamma)$ is their shared geometric center and ω is the new hyper-SD controlling how much the group log scales vary.

Apply the earlier sum-to-zero shift to the log-SD deviations:

$$\begin{aligned} \bar{b} &= J^{-1} \sum_j b_j, & b_j^{\text{s2z}} &= b_j - \bar{b}, & \gamma_{\text{s2z}} &= \gamma + \bar{b}, \\ \log s_j &= \gamma + b_j = \gamma_{\text{s2z}} + b_j^{\text{s2z}}. \end{aligned}$$

Different variances on the varying intercepts

The statistical model uses the nonnegative hyper-SD ω :

$$\gamma \sim \mathcal{N}(\mu_\gamma, s_\gamma^2), \quad b_j \mid \omega \sim \mathcal{N}(0, \omega^2), \quad \log s_j = \gamma + b_j.$$

ω is the sampled magnitude, coded as the signed form λ_ω to preserve continuity at zero.

$$\lambda_\omega \sim \mathcal{N}(0, 0.25^2), \quad \omega = |\lambda_\omega|, \\ z_j \sim \mathcal{N}(0, 1), \quad b_j = \lambda_\omega z_j.$$

Because z_j is symmetric, $\lambda_\omega = +\omega$ and $\lambda_\omega = -\omega$ induce the same $b_j \mid \omega \sim \mathcal{N}(0, \omega^2)$.

$$\sum_j z_j^{s_{2z}} = 0, \quad b_j^{s_{2z}} = \lambda_\omega z_j^{s_{2z}}, \quad \gamma_{s_{2z}} = \gamma + \bar{b}.$$

The removed mean has variance $\lambda_\omega^2/J = \omega^2/J$, so

$$\boxed{\text{Var}(\gamma_{s_{2z}} \mid \lambda_\omega) = s_\gamma^2 + \frac{\lambda_\omega^2}{J} = s_\gamma^2 + \frac{\omega^2}{J}}.$$

Recovery

Recovering the removed log-scale mean $\bar{b}$

Let $v_b = \lambda_\omega^2/J = \omega^2/J$ and $V = s_\gamma^2 + v_b$. Then

$$\bar{b} \mid \gamma_{s2z}, \lambda_\omega \sim \mathcal{N}\left(\frac{v_b}{V}(\gamma_{s2z} - \mu_\gamma), \frac{s_\gamma^2 v_b}{V}\right),$$

where the second argument is a variance. Using $z_{\bar{b}} \sim \mathcal{N}(0, 1)$, recover

$$\begin{aligned} \bar{b} &= \frac{v_b}{V}(\gamma_{s2z} - \mu_\gamma) + \lambda_\omega \sqrt{\frac{s_\gamma^2}{JV}} z_{\bar{b}}, \\ \gamma &= \gamma_{s2z} - \bar{b}, \quad b_j = b_j^{s2z} + \bar{b}. \end{aligned}$$

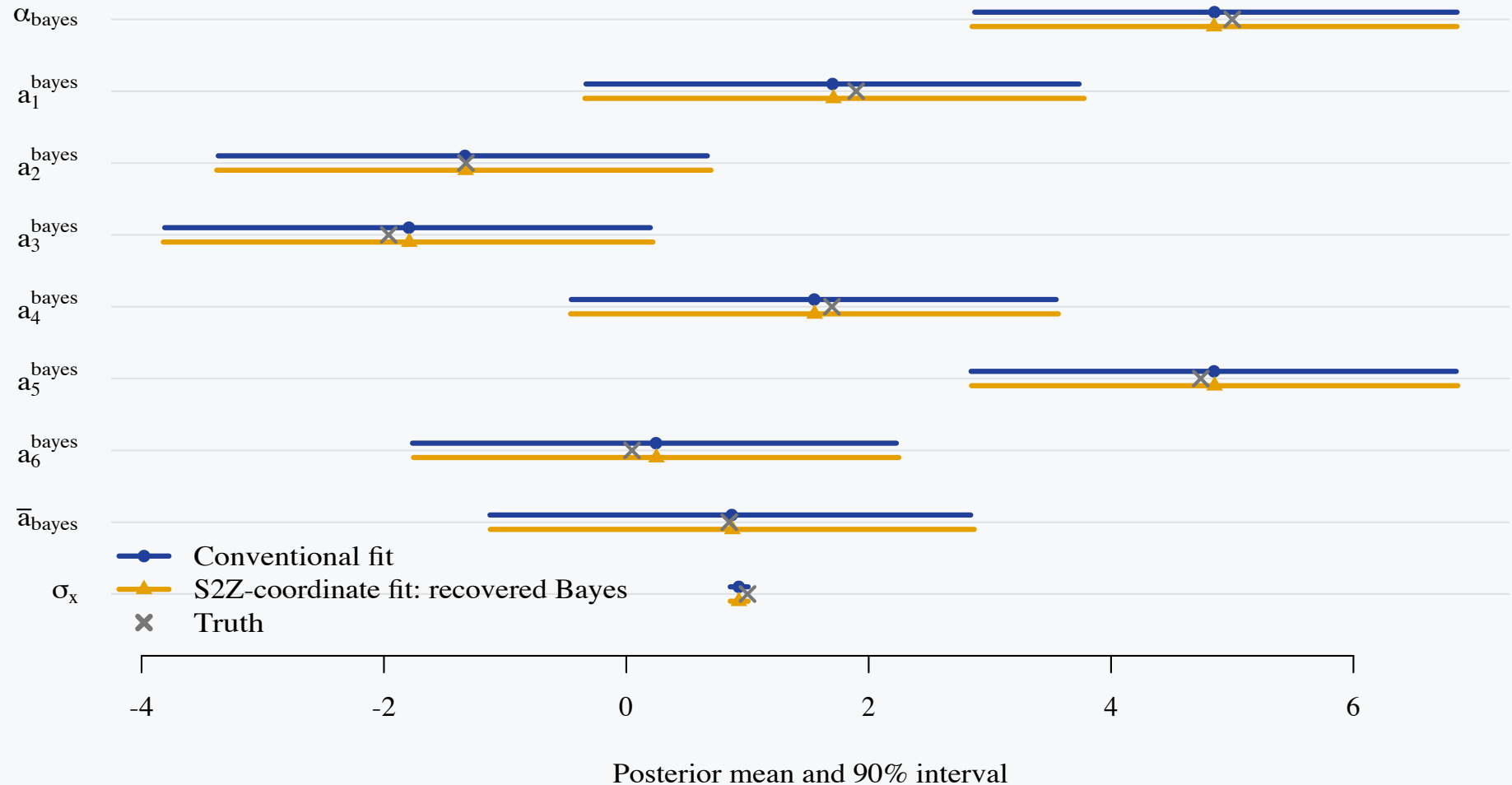
Recovering the removed varying-effect mean m

Let $m = \bar{a}_{\text{bayes}} = J^{-1} \sum_j a_j^{\text{bayes}}$ be the mean removed from the unconstrained Bayesian effects. Given $s_\alpha, s_j, \alpha_{s2z}$, and a^{s2z} , define

$$P = s_\alpha^{-2} + \sum_j s_j^{-2}, \quad \widehat{m} = P^{-1} \left(\frac{\alpha_{s2z}}{s_\alpha^2} - \sum_j \frac{a_j^{s2z}}{s_j^2} \right).$$

Then $m \mid \alpha_{s2z}, a^{s2z}, s_\alpha, \{s_j\} \sim \mathcal{N}(\widehat{m}, P^{-1})$ and $\alpha_{\text{bayes}} = \alpha_{s2z} - m, a_j^{\text{bayes}} = a_j^{s2z} + m$.

Different variances: the two fits agree



The shared log scale has a $\mathcal{N}(\log 2.5, 0.5^2)$ prior, and $\lambda_\omega \sim \text{Normal}(0, 0.25)$. Conditional on them, $s_j = \exp(\gamma + b_j)$ with $b_j \sim \mathcal{N}(0, \omega^2)$.

Conventional versus S2Z efficiency

Metric	Conventional	S2Z coordinates
Post-warmup leapfrog steps	4,051,292	284,280
Wall-clock fit time	4.2 s	0.7 s
Bulk ESS for α	8,108	58,604
Bulk ESS/grad for α	0.00200	0.20615

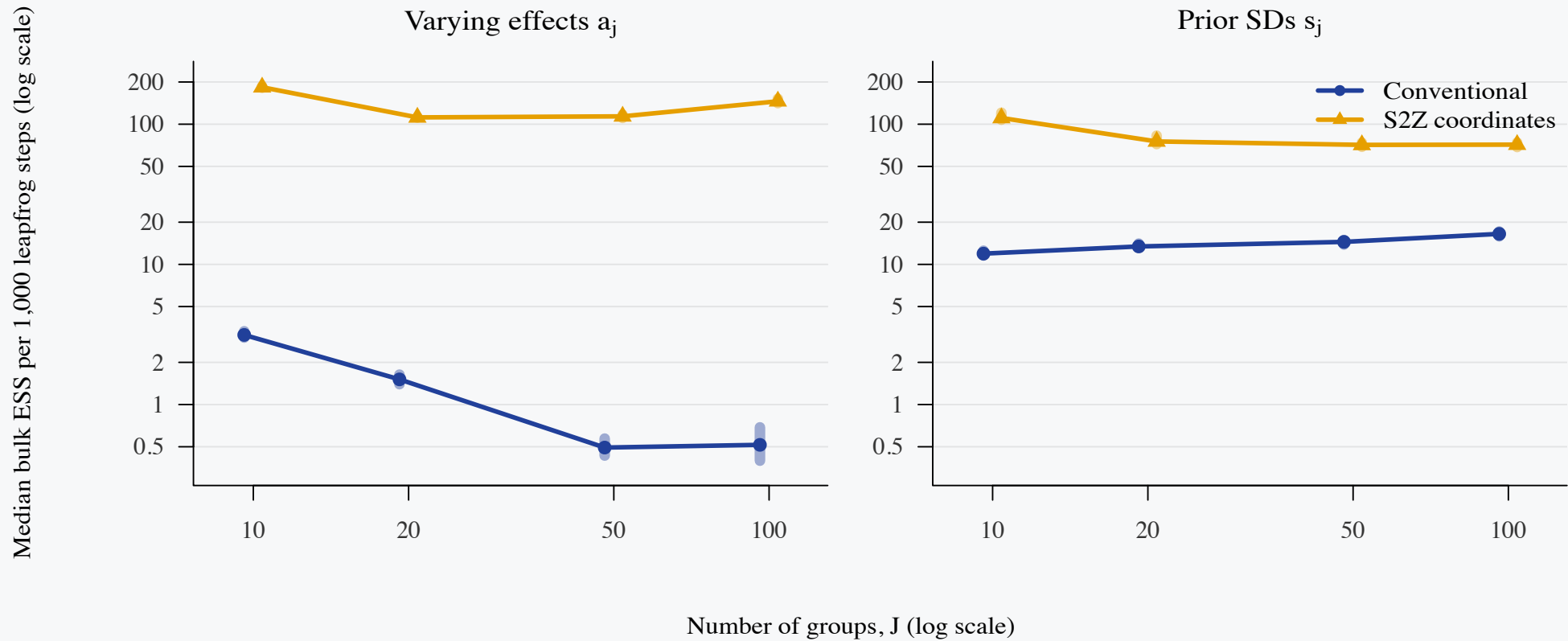
Both fits had zero divergences and zero maximum-treedepth hits. Wall-clock time is CmdStanR's total elapsed fit time for four parallel chains, including warmup and sampling but excluding compilation; it is machine-specific.

Sampling efficiency: effects and SDs

Parameter	Conventional		S2Z coordinates	
	ESS	ESS/grad	ESS	ESS/grad
α	8,108	0.00200	58,604	0.20615
a_j (mean)	8,264	0.00204	58,463	0.20565
$\bar{a}$	8,110	0.00200	58,919	0.20726
σ_x	39,248	0.00969	67,397	0.23708
s_α	27,305	0.00674	60,411	0.21251
s_j (mean)	25,778	0.00636	37,942	0.13347
λ_ω	27,275	0.00673	50,933	0.17916

The a_j and s_j rows report means across the six group-specific parameters; their individual ESS and ESS/grad values had similar magnitudes. Matched Bayesian estimands use 40,000 post-warmup draws. ESS/grad divides by total post-warmup `n_leapfrog__`: conventional = 4,051,292; S2Z coordinates = 284,280. s_α is the population-intercept prior SD, s_j is the prior SD of varying intercept a_j , and ω is the SD of the group log-SD deviations.

ESS per gradient as groups grow



This single nested benchmark extends the original $\tau = 2.5$ simulation by repeating its six unbalanced group sizes through $J = (10, 20, 50, 100)$, yielding $N = (296; 681; 1,781; 3,596)$. Points show median ESS/grad across the matched group-specific estimands; bars show their IQR, not uncertainty intervals.

Take-away

Bayesian hierarchical models allow estimating a 'super-population' but it's not magic, it is a consequence of the distributional assumptions.

Sum-to-zero constraints are great and now you can get the Bayesian estimates with faster and more robust fitting.

Arguments here extend to elliptically symmetric distributions.

More details to come in the paper

- Multiple sets of varying effects
- Varying slopes conversions

Thank you!



github.com/spinkney/open-talks

Appendix

Comparison diagnostics, Stan implementations, and Gaussian marginalization details

On the difference between two posterior means

For each plotted parameter, calculate

$$z = \frac{|\hat{\mu}_1 - \hat{\mu}_2|}{\sqrt{\text{MCSE}_1^2 + \text{MCSE}_2^2}}.$$

Fit comparison	Largest absolute posterior-mean difference	Largest standardized gap, z
Recovered S2Z vs native S2Z	0.003	1.8
Native Bayesian vs recovered Bayesian	0.032	1.6
Different variances: conventional vs recovered Bayesian	0.008	0.5

Each entry is the maximum across the plotted parameters for that comparison; the two maxima need not occur for the same parameter. The largest standardized gap was 1.8 combined Monte Carlo SEs. For one standard-normal comparison, a gap at least this large occurs about 7% of the time; taking a maximum over several parameters makes it still less surprising.

Differing variances conventional Bayesian Stan model: declarations

```
1 data {  
2   int<lower=1> N;  
3   int<lower=2> J;  
4   array[N] int<lower=1, upper=J> group;  
5   vector[N] x;  
6 }  
7 transformed data {  
8   real mu_gamma = log(2.5);  
9   real s_gamma = 0.5;  
10 }  
11 parameters {  
12   real alpha_bayes;  
13   vector[J] a_bayes;  
14   real gamma;  
15   vector[J] z_log_sd;  
16   real lambda_omega;  
17   real<lower=0> sigma_x;  
18 }
```

Differing variances conventional Bayesian Stan model: SD hierarchy and model

```
18 }
19 transformed parameters {
20   real<lower=0> alpha_prior_sd = exp(gamma);
21   vector[J] b = lambda_omega * z_log_sd;
22   vector<lower=0>[J] a_prior_sd = exp(gamma + b);
23 }
24 model {
25   gamma ~ normal(mu_gamma, s_gamma);
26   z_log_sd ~ std_normal();
27   lambda_omega ~ normal(0, 0.25);
28   alpha_bayes ~ normal(0, alpha_prior_sd);
29   a_bayes ~ normal(0, a_prior_sd);
30   sigma_x ~ normal(0, 2);
31   x ~ normal(alpha_bayes + a_bayes[group], sigma_x);
32 }
33 generated quantities {
34   real mean_a = mean(a_bayes);
35   real alpha_s2z_recovered = alpha_bayes + mean_a;
36   vector[J] a_s2z_recovered = a_bayes - mean_a;
37 }
```

S2Z Stan: declarations

```
1 data {  
2   int<lower=1> N;  
3   int<lower=2> J;  
4   array[N] int<lower=1, upper=J> group;  
5   vector[N] x;  
6 }  
7 transformed data {  
8   real mu_gamma = log(2.5);  
9   real s_gamma = 0.5;  
10 }  
11 parameters {  
12   real alpha_s2z;  
13   sum_to_zero_vector[J] a_s2z;  
14   real z_mean_a_cond;  
15   real gamma_s2z;  
16   sum_to_zero_vector[J] z_log_sd_s2z;  
17   real z_mean_b_cond;  
18   real lambda_omega;  
19   real<lower=0> sigma_x;  
20 }
```

S2Z Stan: log-scale constraint

```
21 transformed parameters {
22   real var_gamma = square(s_gamma);
23   real var_mean_b = square(lambda_omega) / J;
24   real var_gamma_s2z = var_gamma + var_mean_b;
25   real mean_b_cond = var_mean_b / var_gamma_s2z
26     * (gamma_s2z - mu_gamma);
27   real mean_b = mean_b_cond + lambda_omega
28     * sqrt(var_gamma / (J * var_gamma_s2z))
29     * z_mean_b_cond;
30   real gamma_recovered = gamma_s2z - mean_b;
31   real<lower=0> alpha_prior_sd_recovered =
32     exp(gamma_recovered);
33   vector[J] b_s2z = lambda_omega * z_log_sd_s2z;
34   vector[J] b_recovered = b_s2z + mean_b;
35   vector<lower=0>[J] a_prior_sd =
36     exp(gamma_s2z + b_s2z);
37   vector[J] inv_var_a = inv(square(a_prior_sd));
```

S2Z Stan: remove the effect mean

```
38   real inv_var_alpha =  
39     inv(square(alpha_prior_sd_recovered));  
40   real precision_mean_a = inv_var_alpha + sum(inv_var_a);  
41   real mean_a_cond = (alpha_s2z * inv_var_alpha  
42     - dot_product(a_s2z, inv_var_a)) / precision_mean_a;  
43   real sd_mean_a_cond = inv_sqrt(precision_mean_a);  
44   real mean_a_bayes = mean_a_cond + sd_mean_a_cond * z_mean_a_cond;  
45   real alpha_bayes_recovered = alpha_s2z - mean_a_bayes;  
46   vector[J] a_bayes_recovered = a_s2z + mean_a_bayes;  
47 }  
48 model {
```

S2Z Stan: prior and likelihood

```
48 model {
49   gamma_s2z ~ normal(mu_gamma, sqrt(var_gamma_s2z));
50   z_log_sd_s2z ~ std_normal();
51   z_mean_b_cond ~ std_normal();
52   lambda_omega ~ normal(0, 0.25);
53
54   alpha_s2z - mean_a_cond ~ normal(0, alpha_prior_sd_recovered);
55   a_s2z + mean_a_cond ~ normal(0, a_prior_sd);
56   target += 0.5 * log(2 * pi()) - 0.5 * log(precision_mean_a);
57   z_mean_a_cond ~ std_normal();
58   sigma_x ~ normal(0, 2);
59   x ~ normal(alpha_s2z + a_s2z[group], sigma_x);
60 }
```

Completing the square in the removed mean m

For fixed s_α , $\{s_j\}$, α_{s2z} , and a^{s2z} , write

$$\ell(m) = \log \mathcal{N}(\alpha_{s2z} - m \mid 0, s_\alpha^2) + \sum_{j=1}^J \log \mathcal{N}(a_j^{s2z} + m \mid 0, s_j^2).$$

With P and $\widehat{m}$ from the main presentation, completing the square gives

$$\ell(m) = \ell(\widehat{m}) - \frac{P}{2}(m - \widehat{m})^2.$$

This immediately gives $m \mid \alpha_{s2z}, a^{s2z}, s_\alpha, \{s_j\} \sim \mathcal{N}(\widehat{m}, P^{-1})$.

Marginalizing the removed mean m

From the completed square, $\ell(m) = \ell(\widehat{m}) - \frac{1}{2}P(m - \widehat{m})^2$. Therefore, marginalizing the removed direction contributes

$$\begin{aligned}\log \int_{-\infty}^{\infty} e^{\ell(m)} dm &= \ell(\widehat{m}) + \log \int_{-\infty}^{\infty} e^{-\frac{P}{2}(m - \widehat{m})^2} dm \\ &= \ell(\widehat{m}) + \frac{1}{2} \log(2\pi) - \frac{1}{2} \log P.\end{aligned}$$

$$\Delta \log p = \frac{1}{2} \log(2\pi) - \frac{1}{2} \log P$$

Thus $P = \text{precision_mean_a}$. This is a Gaussian integration factor, not a Jacobian. The $-\frac{1}{2} \log P$ term must remain because P depends on the sampled scales; $\frac{1}{2} \log(2\pi)$ is constant and may be dropped. If m were sampled directly, this marginalization term would not be added.

Distributions

We will restrict ourselves to elliptically symmetric distributions for hierarchical models, i.e.,

$$f(x) = c g(x^\top \Sigma_x^{-1} x),$$

$$\int_{\mathbb{R}^d} f(x) dx = 1$$

where¹

- the generator $g(\cdot) : \mathbb{R}^+ \rightarrow \mathbb{R}^+$
- the normalizing constant c ensures $f(x)$ integrates to 1

Distributions

$$f(x) = c g(x^\top \Sigma_x^{-1} x),$$
$$\int_{\mathbb{R}^d} f(x) dx = 1$$

The choice of g controls the tail behavior, allowing both light- and heavy-tailed priors within the same family:

- Normal: $g(t) \propto e^{-t/2}$
- Student- t_ν (Cauchy at $\nu = 1$): $g(t) \propto (1 + t/\nu)^{-(\nu+d)/2}$
- Logistic: $g(t) \propto e^{-t} / (1 + e^{-t})^2$
- Laplace: $g(t) \propto e^{-\sqrt{t}}$

References

Delmas, Jean-Pierre. 2024. “Background on Real and Complex Elliptically Symmetric Distributions.” In *Elliptically Symmetric Distributions in Signal Processing and Machine Learning*, edited by Jean-Pierre Delmas, Mohammed Nabil El Korso, Stefano Fortunati, and Frédéric Pascal, 1–34. Cham: Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-52116-4_1.